```
================================================================================
TXT2PDF V1.7.2 - THE SPARTAN PDF GENERATOR
Zero-dependency TXT to PDF converter in pure C11
================================================================================

"Turn plain text into structured PDFs without asking for permission."

Language:   C11 (pure, no C++ runtime, no external libs)
Binary:     Static, stripped, musl libc, tiny
Output:     PDF 1.4 with outlines, internal links, Base14 fonts,
            optional embedded Unicode TrueType fonts, images
Status:     v1.7.2 feature release

================================================================================
QUICK START
================================================================================

- Compile with the provided script:

    ./compile.sh

- Compile manually with musl:

    musl-gcc -Os -Wall -Wextra -std=c11 -static -o txt2pdf txt2pdf.c
    strip txt2pdf

- Run with default Courier font:

    ./txt2pdf readme.txt readme.pdf

- Run with Helvetica:

    ./txt2pdf --sans readme.txt readme.pdf

- Run with Times-Roman:

    ./txt2pdf --serif readme.txt readme.pdf

- Run with an embedded Unicode TrueType font:

    ./txt2pdf --font NotoSansJP-Regular.ttf readme.txt readme.pdf

- Run with debug diagnostics:

    ./txt2pdf --debug readme.txt readme.pdf

================================================================================
[TOC] TABLE OF CONTENTS
================================================================================

1.  what is txt2pdf?
2.  what is new in v1.7.2?
3.  fonts and typography
4.  image support
5.  parser rules
6.  table of contents rules
7.  usage and debug mode
8.  minimal working structure
9.  pdf internals
10. project structure
11. building and static cookbook
```

12. known limitations
13. troubleshooting
14. design philosophy
15. license and credits

==========
1. WHAT IS TXT2PDF?
==========

txt2pdf is a zero-dependency command-line utility that converts plain text files into structured PDF 1.4 documents.

Unlike Pandoc, wkhtmltopdf, or Python-based converters that require hundreds of megabytes of runtime dependencies, txt2pdf is a small static binary.

It parses text, calculates page breaks, generates PDF objects, and writes the final binary stream directly to disk using only the C standard library.

It is designed for environments where installing dependencies is impossible, unwanted, or where the network is down.

==========
2. WHAT IS NEW IN V1.7.2?
==========

Version 1.7.2 adds optional Unicode font support for Japanese, Chinese, Korean and other scripts covered by the selected TrueType font.

Main changes:

- Added --font FILE.ttf mode.
- Added embedded TrueType font support.
- Added Unicode CJK output using PDF Type0 / CIDFontType2.
- Added /Identity-H encoding.
- Added /ToUnicode maps for copy, paste and search.
- Added real glyph metrics from the TTF for line wrapping.
- Added UTF-16BE bookmark titles for CJK outlines.
- Added automatic TrueType subsetting.
- Kept the classic Base14 mode unchanged when --font is not used.

Example size impact:

- Full Noto Sans JP embedded: ~5 MB PDF
- Same document with v1.7.2 subsetting: ~75 KB PDF

The default behavior remains unchanged:

- Courier is still the default font.
- Existing v1.7.1 documents convert exactly as before.
- CJK mode and subsetting are opt-in features.

==========
3. FONTS AND TYPOGRAPHY
==========

txt2pdf v1.7.2 supports two font pipelines.

A. Base14 fonts (classic mode):

    --mono   Courier (default, classic txt2pdf layout)
    --sans   Helvetica (clean, modern, proportional)

```
    --serif    Times-Roman (classic documentation look)

    Examples:

        ./txt2pdf --mono input.txt output.pdf
        ./txt2pdf --sans input.txt output.pdf
        ./txt2pdf --serif input.txt output.pdf

    Important notes:

    - Base14 fonts are not embedded into the PDF.
    - PDF viewers normally provide built-in substitutes.
    - Courier layout remains fixed-width and highly predictable.
    - Helvetica and Times-Roman use built-in ASCII metrics for wrapping.
    - Accents and non-ASCII characters remain limited to WinAnsiEncoding.

B. Embedded TrueType font (CJK mode):

    --font FILE.ttf

    Examples:

        ./txt2pdf --font NotoSansJP-Regular.ttf roadmap.txt roadmap.pdf
        ./txt2pdf --font NotoSansSC-Regular.ttf input.txt output.pdf
        ./txt2pdf --font NotoSansKR-Regular.ttf input.txt output.pdf

    Important notes:

    - The font must be an individual TrueType .ttf file.
    - OpenType/CFF .otf files are not supported yet.
    - TrueType collections .ttc are not supported yet.
    - When --font is used, it overrides --sans, --serif and --mono.
    - The selected font is used for ALL text (ASCII, Latin, CJK).
    - Only the glyphs actually used in the document are embedded.

================================================================================
4. IMAGE SUPPORT
================================================================================

txt2pdf v1.7 supports standalone block images.

Supported image tags:

    [IMG]photo.jpg[/IMG]
    [IMG]diagram.jpeg[/IMG]
    [IMG]chart.png[/IMG]

The image line must be the only content on that line.

Good image line:

    [IMG]chart.png[/IMG]

Bad image line:

    See this chart: [IMG]chart.png[/IMG]

Supported formats:

- JPEG: Embedded using PDF native DCTDecode passthrough.
- PNG: Embedded using a minimalist passthrough for simple PNG files.
```

Supported PNG types:

- 8-bit grayscale
- 8-bit RGB
- 8-bit palette

Unsupported PNG types:

- alpha transparency
- interlaced PNGs
- 16-bit channels
- palette transparency

Image layout behavior:

- Images are treated as standalone content blocks.
- Aspect ratio is preserved.
- Images wider than the text area are scaled down.
- Images taller than the page are scaled down to fit.
- An image may force a page break if it does not fit.
- Image paths are relative to the current working directory.
- The PDF stores the original compressed image data.
- Reducing source image size reduces final PDF size.

==
5. PARSER RULES
==

The parser creates PDF bookmarks only for lines that are real section
headers.

A line is considered a section header only if all of these conditions are
true:

- It starts with a digit, optionally followed by letters, then a dot.
- The title text after the dot contains only uppercase letters.
- The line is surrounded by separator lines made mostly of '=' characters.
- The line is not inside a contents block.

Valid section shape:

- separator line
- uppercase numbered title
- separator line

Rejected lines:

- numbered titles containing lowercase letters
- numbered titles used as examples inside normal paragraphs
- numbered titles not surrounded by separator lines
- numbered titles inside the contents block

This makes the parser much safer for manuals that contain examples.

The parser supports hierarchical IDs such as:

- 1
- 2.3
- 2.3.1
- 11.14

- 2.0a
- 13.1b
- 26.5a

==
6. TABLE OF CONTENTS RULES
==

Internal links are created only from contents blocks.

A contents block is detected when:

- a line contains the token "[TOC]"
- the next non-blank line is a separator line
- the block ends at the next separator line

Rules for entries:

- entries must be numbered lines
- lowercase entries are recommended
- entries are not converted into bookmarks
- entries are linked to the matching section by number or title
- uppercase entries inside the contents block are also ignored as bookmarks

This means the old duplicate-bookmark problem is largely eliminated by the
parser itself.

==
7. USAGE AND DEBUG MODE
==

Basic usage:

```
./txt2pdf input.txt output.pdf
```

Font selection:

```
./txt2pdf --mono input.txt output.pdf
./txt2pdf --sans input.txt output.pdf
./txt2pdf --serif input.txt output.pdf
./txt2pdf --font FILE.ttf input.txt output.pdf
```

Debug usage:

```
./txt2pdf --debug input.txt output.pdf
```

Combined usage:

```
./txt2pdf --debug --sans input.txt output.pdf
./txt2pdf --debug --font NotoSansJP-Regular.ttf input.txt output.pdf
```

Debug mode prints diagnostics to stderr.

It reports:

- detected encoding
- selected font
- lines marked as part of a contents block
- detected section headers
- skipped uppercase numbered lines
- detected internal links

- detected images
- image placements
- CJK glyph count and subset size (in CJK mode)

Use debug mode when:

- a bookmark is missing
- too many bookmarks appear
- a contents entry does not link anywhere
- an image does not appear
- CJK characters render as boxes or ?
- you are adapting a new readme to the Spartan format

==
8. MINIMAL WORKING STRUCTURE
==

A minimal compatible document needs three things:

- a contents block starting with the [TOC] token
- lowercase or mixed-case contents entries
- uppercase section headers surrounded by separator lines

Recommended structure:

- write the contents title line containing [TOC]
- write a separator line
- write entries such as "1. introduction" and "2. conclusion"
- write a separator line
- for each section, write a separator line
- write the uppercase header, for example "1. INTRODUCTION"
- write another separator line
- write the section body

Important:

Do not wrap example headers with separator lines unless you want them to
become real bookmarks.

==
9. PDF INTERNALS
==

txt2pdf generates a valid PDF 1.4 structure from scratch.

Catalog and Pages:

Calculates A4 dimensions, applies margins, wraps text, and places content
using a line-based layout.

Fonts:

Generates a Base14 Type1 font object. The default is Courier. The --sans
and --serif options switch to Helvetica or Times-Roman.

In CJK mode, it generates a Type0 font tree with a CIDFontType2 descendant,
an embedded FontFile2 stream (subsetted TTF), and a ToUnicode CMap.

Images:

Loads image metadata, reserves vertical layout space, writes image XObject

streams, and references them from page resources.

Encoding and Smart Decoder:

Auto-detects UTF-8 vs Latin-1. If UTF-8 is detected, a smart decoder translates supported multi-byte sequences to WinAnsiEncoding.

In CJK mode, UTF-8 codepoints are mapped directly to glyph IDs from the embedded TrueType font, and the content stream uses hexadecimal notation with Identity-H encoding.

Outlines:

Generates a linked list of PDF Outline objects pointing to the exact XYZ coordinates of each uppercase header. In CJK mode, outline titles use UTF-16BE hex encoding so they display CJK characters correctly.

Annotations:

Scans contents blocks and injects link annotations for each entry matching a real section.

Cross-Reference Table:

Calculates byte offsets for every object and writes a compliant xref table and trailer.

10. PROJECT STRUCTURE

The txt2pdf codebase follows a single-file architecture.

Typical layout:

- txt2pdf.c
- compile.sh
- readme.txt
- README.md

The main C file contains:

- argument parsing
- encoding detection
- text parsing
- header detection
- contents detection
- image loading
- font selection (Base14 and TrueType)
- TrueType subsetting
- page break calculation
- PDF object generation
- binary stream writing

The tool is intentionally small and auditable.

11. BUILDING AND STATIC COOKBOOK

Provided build script:

```
    ./compile.sh

Dynamic build:

    gcc -O2 -Wall -Wextra -std=c11 -o txt2pdf txt2pdf.c

Static musl build:

    musl-gcc -Os -Wall -Wextra -std=c11 -static -o txt2pdf txt2pdf.c
    strip txt2pdf

Why musl?

glibc static binaries are often much larger due to NSS and locale data.
musl provides a clean, minimal libc that results in a tiny, fully
self-contained executable.

Verify static build:

    file txt2pdf
    ldd txt2pdf

Expected:

- file reports statically linked
- ldd says it is not a dynamic executable

================================================================================
12. KNOWN LIMITATIONS
================================================================================

General:

- No external URL hyperlinks.
- No custom page sizes.
- No custom margins.
- No multi-column layout.
- No inline images.
- No image captions.
- No repeated header/footer images.
- No per-image width, height, DPI, or centering attributes yet.

Images:

- PNG support is limited to simple 8-bit non-interlaced images.
- PNG alpha transparency is not supported.
- Interlaced PNGs are not supported.

Base14 mode:

- WinAnsiEncoding only.
- Characters outside WinAnsi are replaced with '?'.
- Base14 fonts are not embedded.

CJK mode:

- Requires an individual TrueType .ttf file.
- OpenType/CFF .otf files are not supported yet.
- TrueType collections .ttc are not supported yet.
- No automatic per-script font fallback.
```

- Scripts requiring shaping (Arabic, Indic, Thai) are not supported.

Parser:

- Section headers must be fenced by separator lines.
- Internal links are only generated from contents blocks.

13. TROUBLESHOOTING

Missing bookmarks:

- The header contains lowercase letters.
- The header is not surrounded by separator lines.
- The header is accidentally inside a contents block.

Duplicate bookmarks:

- Old documents may have uppercase contents entries.
- The updated parser ignores contents entries as sections.
- If duplicates persist, run with --debug and inspect skipped lines.

Missing links:

- The entry is not inside a detected contents block.
- The entry number does not match any section.
- The entry title does not match any section title.

Extra bookmarks:

- An example header is surrounded by separator lines.
- Remove the separators around examples or rewrite the example line.

Garbled characters in Base14 mode:

- Ensure your text editor saves the file in UTF-8.
- Run with --debug to verify the detected encoding.
- Characters outside WinAnsiEncoding cannot be rendered.
- Use --font FILE.ttf for full Unicode support.

CJK characters appear as boxes or '?' in CJK mode:

- The .ttf file does not contain the requested glyphs.
- Use a font with broader coverage (e.g. Noto Sans SC for Simplified
  Chinese, Noto Sans TC for Traditional Chinese, Noto Sans JP for
  Japanese, Noto Sans KR for Korean).
- Check that the file is a real .ttf, not .otf or .ttc.

Cannot load TTF font error:

- The file is not in the current directory. Use absolute path.
- The file is .otf (CFF) instead of .ttf (TrueType).
- The file is a .ttc collection (not supported yet).
- The file is corrupted or incomplete.

Image does not appear:

- The file path is wrong.
- The image line contains extra text beside the tag.
- The image format is not supported.

- The PNG uses alpha, interlacing, or 16-bit channels.

Image is too large:

- v1.7 does not yet support per-image width or DPI attributes.
- Use a smaller source image or crop it.
- High-resolution images are scaled visually but still embedded.

Font does not look as expected:

- Use --mono for the classic Courier layout.
- Use --sans for Helvetica.
- Use --serif for Times-Roman.
- Use --font FILE.ttf for Unicode CJK layout.
- Some viewers substitute Base14 fonts with similar fonts.

Debug output is the fastest way to diagnose the problem.

14. DESIGN PHILOSOPHY

Modern software development has forgotten how to build small tools.

We wrap simple text transformations in Electron apps, Node.js runtimes,
and Docker containers.

txt2pdf is a rejection of that bloat.

It proves that a fully functional, structured document generator can fit
in a tiny static binary, written in pure C11, with zero external
dependencies.

When the cloud goes down, and the package managers fail to resolve,
the Spartan tool remains.

"Code is neither created nor destroyed; it is distilled."

15. LICENSE AND CREDITS

Source code : GPLv3
Manual: GFDL v1.3

txt2pdf was written as a demonstration of minimalist software design.

Inspired by the Unix philosophy: do one thing well.

Built with minimalism, on a potato, for potatoes.

END OF DOCUMENTATION